

★ WEEK 6 · PROBLEM SET

Finance Applications: Three Hands-On Problems

The Widest Road · Soft-Margin Slack · RBF Similarity

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

Small numbers you can compute with a pencil — then a metaprompt that lets Claude Code scale the same idea to real data. Hand first, machine second.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the prompts generate or fetch free data; CSVs are hand-calc references

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Widest road	§ 5 in-script set (make_classification, fixed seed)	fml_w6_p1_scores.csv (hand-calc)
P2 · Soft margin	§ 5 in-script set — same data, one C sweep	fml_w6_p1_scores.csv (incl. the outlier)
P3 · RBF similarity	§ 5 in-script set + OpenML 'creditcard' (optional)	fml_w6_p3_distances.csv (hand-calc)
Real-data extension	sklearn fetch_openml('creditcard') — free, no key	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · THE MARGIN

Find the Widest Road by Hand

Five credit scores on a line — one safest boundary, two support vectors

Five Customers on a Risk Line

Repaid: scores 2, 3, 4. Defaulted: scores 8, 9. Draw the boundary an SVM would draw

Customer	R1	R2	R3	D1	D2
Risk score	2	3	4	8	9
Outcome	repaid	repaid	repaid	default	default

The rule

- One number b : predict RISKY if score $> b$.
- Any b that separates the groups is 'correct' on this data.
- SVM asks for more: the SAFEST such b .

Your deliverables

- The feasible range of b , the SVM boundary b^* , and the margin.
- The two support vectors.
- Stress test: new repaid at 4.8 — verdict under $b^* = 6$ vs $b = 4.5$?

The Tools: A Feasible Range and Its Middle

In 1-D the whole SVM collapses to one midpoint — the picture survives to any dimension

rule: predict RISKY if score $> b$ feasible $b \in (4, 8)$ — SVM picks the middle

Why the middle

- The nearest repaid customer sits at 4; the nearest defaulter at 8.
- Any b near 4 or 8 hugs one group — one noisy arrival breaks it.
- The midpoint maximizes the distance to BOTH nearest points.

Lab guide — three steps

- Step 1: find the inner edge of each group (4 and 8).
- Step 2: b^* = their midpoint; margin = half the gap.
- Step 3: name the support vectors — the edge points only.

Midpoint, Margin, and a Stress Test

The hugging boundary fails the first noisy arrival; the SVM boundary shrugs it off

Question	Arithmetic	Answer
Feasible boundaries	between $\max(\text{repaid}) = 4$ and $\min(\text{default}) = 8$	$b \in (4, 8)$
SVM boundary b^*	$(4 + 8) / 2$	$b^* = 6$
Margin (buffer each side)	$(8 - 4) / 2$	2 score points
Support vectors	the two inner edges only	R3 (4) and D1 (8)
New repaid arrival at 4.8	vs $b^* = 6$: $4.8 < 6$ ✓ safe · vs hugging $b = 4.5$: $4.8 > 4.5$ ✗ flagged	the wide road absorbs the noise

$$b^* = \frac{4+8}{2} = 6 \quad \text{margin} = \frac{8-4}{2} = 2 \quad \text{support vectors: 4 and 8}$$

$b^* = 6$, Margin 2 — and Only Two Points Matter

Move R1 from 2 to 0: nothing changes. Move R3 from 4 to 5: everything changes

$$b^* = 6$$

The one boundary
that hugs neither
group

$$\pm 2$$

Buffer on each side —
noise insurance,
prepaid

$$2 / 5$$

Support vectors — the
only customers the
model remembers

$$4.8 \checkmark$$

The noisy arrival
survives $b^* = 6$, dies at
 $b = 4.5$

Week 1, again

- The wide road is variance reduction made visible.
- A hugging boundary is Student A drawing the class line.

What survives in high-D

- Same story with w and $1/||w||$ instead of a midpoint.
- Support vectors still carry the model — the rest is décor.

Scale It: See the Support Vectors on Real Data

Paste the metaprompt — Claude Code fits an SVM and shows you who carries it

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42)  
   and takes the first 4000 rows (SVM is slow);  
2) standardizes features and fits a linear SVC (C=1);  
3) prints: number of support vectors per class, and  
   the share of training points they represent;  
4) proves the P1 lesson: refit on the data WITHOUT  
   the 500 points farthest from the boundary and show  
   the test F1 barely moves; then drop 50 support  
   vectors instead and show the boundary shifts;  
5) two-line verdict: who carries an SVM, and why that  
   makes it memory-light (Week 6 language).
```

[OUTPUT] What to expect

```
support vectors: a small  
  minority of the rows  
dropping far points: F1  
  almost unchanged  
dropping SVs: the boundary  
  and F1 visibly move  
  
Your 2-of-5 lesson, at  
4,000-row scale.
```

P2

PROBLEM 2 · SOFT MARGIN

One Oddball, Two Philosophies

A weird-but-repaid customer at score 7 — hard vs soft, decided by C

The Outlier Walks In

Same five customers as P1 — plus R4: risk score 7, yet repaid. Rebuild the road twice

Customer	R1	R2	R3	R4 (new)	D1	D2
Risk score	2	3	4	7	8	9
Outcome	repaid	repaid	repaid	repaid (!)	default	default

The two philosophies

- HARD margin: every point must be on its own side of the road.
- SOFT margin: a point may cross, paying slack ξ = how far it sits past its side's edge.

Your deliverables

- Hard-margin boundary, margin, fragility.
- Soft option: keep $b = 6$ — compute R4's slack ξ .
- Verdict per C: who wins when C is huge? tiny?

The Tool: A Price Tag on Rule-Breaking

The objective buys road width and pays for slack — C is the exchange rate

$$\min \underbrace{\frac{1}{\text{margin}}}_{\text{narrow road cost}} + C \cdot \underbrace{\sum_i \xi_i}_{\text{rule-breaker cost}} \quad \xi_7 = 7 - 4 = 3$$

Reading the trade

- Wide road $\rightarrow$ first term small; slack $\rightarrow$ second term grows.
- R4 at 7 sits past the repaid edge (4) by 3 $\rightarrow \xi = 3$.
- C converts slack into cost — the only judge of the trade.

Lab guide — two builds

- Build A (hard): squeeze the road between 7 and 8.
- Build B (soft): keep P1's road, bill R4 for $\xi = 3$.
- Compare margins; state which C regime picks which.

Two Roads, Priced Side by Side

Hard buys zero slack with a 4× narrower road; soft pays $\xi = 3$ and keeps the highway

	Build A — hard margin	Build B — soft margin
Boundary b	$(7 + 8) / 2 = 7.5$	stays at 6 (ignore R4)
Margin	$(8 - 7) / 2 = 0.5$	$(8 - 4) / 2 = 2.0$
Slack paid	0	$\xi = 7 - 4 = 3$ (one customer)
Character	obeys everyone, trusts no buffer	forgives one oddball, keeps the buffer
A new repaid at 6.8?	$6.8 < 7.5 \checkmark$ — barely survives	$6.8 > 6 \times$ flagged — the price of forgiving R4

hard: $b = 7.5$, margin 0.5, $\xi = 0$

soft: $b = 6$, margin 2, $\xi = 3$

C Is the Judge — Not the Data

Same six customers; huge C builds the brittle road, small C builds the forgiving one

0.5

Hard margin — one oddball crushed the road 4×

$\xi = 3$

Soft margin's bill for ignoring R4

C ↑

Slack unaffordable → Build A wins → overfit risk

C ↓

Slack cheap → Build B wins → robust road

The honest trade

- Neither build is 'right' — they price mistakes differently.
- Financial data is full of R4s: weird-but-honest cases.

Week 4's cousin

- Big C $\approx$ small λ : little regularization, maximal obedience.
- The same U-curve — swept on validation, never on test.

Scale It: Sweep C and Watch the Road Breathe

Paste the metaprompt — Claude Code prices the trade across four orders of magnitude

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) generates the class dataset (`make_classification`, 20000 x 20, `weights=[0.99, 0.01]`, `random_state=42`), takes 4000 rows, standardizes, splits train/val/test;
- 2) fits linear SVC for C in $[0.01, 0.1, 1, 10, 100]$ and records for each: validation F1, number of support vectors, and total slack (sum of hinge violations on the training set);
- 3) plots F1, SV count and total slack vs $\log C$ on aligned axes;
- 4) picks C at the validation peak and quotes test F1 once;
- 5) three-sentence verdict: what happened to slack and SV count as C grew, in P2's hard-vs-soft language.

[OUTPUT] What to expect

C tiny: many SVs, big slack, forgiving road
 C huge: fewer violations, brittle fit, val F1 dips
peak F1 at a moderate C

Your two-build table,
priced continuously.

P3

PROBLEM 3 · RBF SIMILARITY

The Fence, Measured in Similarity

Three transactions, two γ settings — who falls outside the fence?

Three Transactions vs. the 'Normal' Prototype

Squared distances given. Compute RBF similarity at $\gamma = 0.5$ and $\gamma = 0.1$; flag when $K < 0.2$

Transaction	d^2 to normal prototype	Story
T1	1	a routine grocery run
T2	4	larger than usual, odd hour
T3	25	foreign, midnight, huge

Helper (e-values)

- $e^{(-0.1)} \approx 0.905$ ·
 $e^{(-0.4)} \approx 0.670$
- $e^{(-0.5)} \approx 0.607$ ·
 $e^{(-2)} \approx 0.135$
- $e^{(-2.5)} \approx 0.082$ ·
 $e^{(-12.5)} \approx$
0.000004

Your deliverables

- The 2×3 similarity table $K = e^{(-\gamma \cdot d^2)}$, the flags at threshold $K < 0.2$ under each γ , and one sentence linking γ to the fence's tightness (OCSVM's eyesight).

The Tool: Similarity That Fades Exponentially

K near 1 = a near twin of normal; K near 0 = a stranger — γ sets how fast twins become strangers

$$K(x, x') = \exp(-\gamma d^2(x, x')) \quad K \approx 1: \text{near twin} \quad K \approx 0: \text{stranger}$$

Why this is the fence

- OCSVM's boundary is built from exactly these K values.
- 'Inside the fence' $\approx$ similar enough to the normal crowd.
- Small γ = long eyesight (loose fence); big γ = tunnel vision (tight fence).

Lab guide — per cell

- One multiplication ($\gamma \cdot d^2$), one e-lookup — six cells total.
- Compare each K to the 0.2 threshold; below = outside the fence.
- Do $\gamma = 0.5$ first, then $\gamma = 0.1$.

Six Cells, Two Fences

Same three transactions — the tight fence flags two, the loose fence flags one

	T1 ($d^2 = 1$)	T2 ($d^2 = 4$)	T3 ($d^2 = 25$)
$\gamma = 0.5: K = e^{(-0.5 \cdot d^2)}$	$e^{(-0.5)} = 0.61 \checkmark$ inside	$e^{(-2)} = 0.14 \times$ FLAG	$e^{(-12.5)} \approx 0.00 \times$ FLAG
$\gamma = 0.1: K = e^{(-0.1 \cdot d^2)}$	$e^{(-0.1)} = 0.90 \checkmark$ inside	$e^{(-0.4)} = 0.67 \checkmark$ inside	$e^{(-2.5)} = 0.08 \times$ FLAG

$$\gamma = 0.5: \{0.61, 0.14, \approx 0\} \quad \gamma = 0.1: \{0.90, 0.67, 0.08\}$$

Threshold $K < 0.2$ = outside the fence. T3 is a stranger to both fences; T2's fate depends entirely on γ .

γ Decided T2's Fate — Not the Data

The tight fence ($\gamma = 0.5$) flags the odd-hour purchase; the loose fence waves it through

0.14 vs 0.67

T2's similarity under $\gamma = 0.5$ vs $\gamma = 0.1$ — same transaction

2 vs 1

Flags raised by the tight fence vs the loose fence

≈ 0.00

T3 under $\gamma = 0.5$ — a stranger by any eyesight

$\gamma + v$

OCSVM's two dials: eyesight and allowed outsiders

The fraud-desk reading

- Tight fence: more catches, more angry honest customers.
- Loose fence: calm alarms, more fraud slips through.

The course's refrain

- γ is Week 5's threshold and Week 4's λ in a new costume.
- Sweep it on validation; let the U-curve pick.

Scale It: Two Fences on Twenty Thousand Rows

Paste the metaprompt — Claude Code sweeps γ and ν and scores every fence

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) generates the class dataset (make_classification,  
   20000 x 20, weights=[0.99, 0.01], random_state=42);  
2) trains OneClassSVM on 4000 NORMAL-only rows for  
   every pair of gamma in [0.001, 0.01, 0.1] x  
   nu in [0.01, 0.02, 0.05];  
3) scores 8000 held-out rows per fence and records:  
   points flagged, real anomalies caught, precision;  
4) prints the 3 x 3 results table and draws a heat map  
   of catches over the (gamma, nu) grid;  
5) optional real-data run: fetch_openml('creditcard')  
   - free, no signup;  
6) three-sentence verdict: which corner is the tight  
   fence, which the loose one, in P3's language.
```

[OUTPUT] What to expect

```
big gamma + big nu: many  
  flags, noisy precision  
small gamma + small nu:  
  quiet fence, misses more  
a sensible middle catches  
  most with modest flags
```

```
Your 6-cell table, run  
9 fences x 8000 rows.
```

Problem Set — Three Things Your Hands Now Know

You computed every number yourself; the code only repeats what you already understand

- 1 The margin: $b^* = 6$, buffer ± 2 , two support vectors**
The middle of the gap survived the noisy 4.8 arrival; the hugging boundary did not.
- 2 Soft margin: pay $\xi = 3$, keep $4 \times$ the road**
One weird-but-repaid customer shrank the hard road to 0.5 — C alone decides whether to forgive.
- 3 RBF: γ set T2's fate**
 $K = 0.14$ vs 0.67 for the same transaction — the fence's tightness is a dial, tuned on validation.

Hand calculation first — then let Claude Code scale the same idea to real data.